Воспроизводимость и оценка эффективности ConvNeXt

Классификация изображений малого разрешения

Промежуточная точка проекта

Анастасия Оленева
НИУ ВШЭ

1. Статья и основная гипотеза

Статья: Feng J. et al. *Enhancing ConvNeXt for Efficient Small-Size Image Classification*, 2025.

Код: `https://github.com/southerly7/Proposed`

Гипотеза. Авторы утверждают, что адаптация архитектуры ConvNeXt к изображениям малого разрешения, в частности использование двухэтапного patch embedding (Twice-Patch Embedding), модифицированных свёрточных блоков и дополнительных механизмов нормализации и внимания, позволяет повысить точность классификации при относительно небольшом количестве параметров.

2. Ядро воспроизведения

Датасет: CIFAR-100, изображения 32×32, 100 классов.

Основная метрика: Top-1 Accuracy (%).

Для воспроизведения выбирается результат предложенной модели из таблицы 3 оригинальной статьи.

Модель	Параметры	Accuracy
ConvNeXt-T	27,8 млн	67,58%
Conv2NeXt-B	7,3 млн	83,82%
Предложенная модель	7,3 млн	85,16%

Основной задачей является воспроизведение точности предложенной модели (85,16%) с использованием авторского кода.

Дополнительно проверяется, насколько результат устойчив к случайной инициализации: планируется выполнить три независимых запуска с различными seed.

Критерий успеха: средняя Top-1 Accuracy отличается от опубликованной не более чем на 1 процентный пункт. Поскольку авторы не приводят стандартное отклонение по seed, этот порог принимается предварительно и будет дополнительно оценён по фактическому разбросу результатов.

3. Запуск оригинального кода

Первый запуск выполнен на основе оригинального репозитория авторов без изменения архитектуры модели и алгоритма обучения. Из-за несовместимости с современными версиями библиотек потребовались небольшие технические исправления.

- Commit hash: 0c57846d62c25ac548d7810d425f00b9299a8e54
- Python: 3.13.16
-
- PyTorch: 2.14.1+cu130
- torchvision: 0.29.1+cu130
- timm: 1.0.31
- CUDA: 13.0
- GPU: NVIDIA Tesla T4

- Время обучения: 5 мин 37 с (≈ 0.094 GPU-ч)
- Полученная accuracy: 11.91% (после 2 эпох)
- Статус: проверочный запуск успешен, потребовались исправления совместимости.

Указанные авторами версии: PyTorch 1.8.0, torchvision 0.9.0, timm 0.3.2.

Для запуска были внесены следующие изменения:

- Удалены импорты недоступных оптимизаторов из `timm` (в эксперименте используется AdamW из PyTorch).
- Устаревший импорт `torch._six.inf` заменён на `math.inf`.
- Установлена дополнительная зависимость `tensorboardX`.
- Добавлена обработка аргументов `pretrained_cfg` и `cache_dir` при создании модели.

Проверочный запуск модели `proposed_base` на CIFAR-100 выполнен с seed 42 в течение двух эпох. Количество параметров модели составило 7.41 млн. Top-1 Accuracy увеличилась с 7.28% до 11.91%, а train loss уменьшился с 4.5738 до 4.3965. Сохранение checkpoint проверено успешно.

Полное обучение на 300 эпох пока не проводилось. Его предварительная вычислительная стоимость оценивается примерно в 14 GPU-часов. Результаты проверочного запуска не сравниваются с заявленной авторами accuracy 85.16%, поскольку использовалось сокращённое обучение без warmup.

4. Проверяемые недостатки

1. Неизолированный вклад модифицированного stem

В статье предлагается адаптированный механизм Twice-Patch Embedding, позволяющий лучше сохранять пространственную информацию при обработке изображений малого разрешения. Однако изменение stem рассматривается вместе с другими модификациями архитектуры, что затрудняет оценку его независимого вклада в итоговую точность.

Проверка: провести абляционный эксперимент, в котором остальные компоненты сети остаются неизменными, а изменяется только входной блок. Планируется сравнить стандартный stem ConvNeXt с ядром \(4\times4\) и шагом 4, адаптированный одноэтапный stem и предложенный авторами Twice-Patch Embedding.

Для каждого варианта будут измерены Top-1 Accuracy, число параметров и FLOPs. Поскольку различные stem могут изменять разрешение промежуточных карт признаков, эти различия будут учитываться при интерпретации результатов.

Ожидание: адаптированный stem обеспечит более высокую точность на CIFAR-100 за счёт сохранения пространственной информации на ранних стадиях. Предполагаемый прирост составляет 1–3 процентных пункта относительно стандартного варианта.

2. Недостаточно строгое сравнение с ResNet

В статье предложенная архитектура сравнивается с несколькими моделями, однако число параметров и условия обучения различаются. Поэтому нельзя однозначно определить, связано ли преимущество с архитектурными решениями или с различиями в вычислительном бюджете и процедуре обучения.

Проверка: обучить ResNet-18 с подобранной шириной каналов таким образом, чтобы число параметров отличалось от предложенной ConvNeXt не более чем на 5%.

Для обеих моделей будут использоваться одинаковые данные, аугментации, оптимизатор, scheduler, количество эпох и протокол оценки. Дополнительно сравниваются FLOPs и время обучения.

Ожидание: улучшенная ConvNeXt сохранит преимущество над ResNet при сопоставимом количестве параметров, однако разница в точности окажется меньше, чем при сравнении моделей в исходной статье.

3. Потенциальная утечка данных при выборе checkpoint

В оригинальной реализации тестовая выборка CIFAR-100 используется для оценки качества после каждой эпохи и выбора лучшего checkpoint. Такой подход может приводить к оптимистичному смещению итоговой метрики, поскольку тестовые данные косвенно участвуют в выборе модели.

Проверка: выделить из обучающей выборки независимый validation set (45 000 изображений для обучения и 5 000 для валидации). Лучший checkpoint будет выбираться исключительно по validation accuracy, а test set использоваться только для итоговой оценки.

Будут сопоставлены результаты оригинальной и исправленной процедур. При анализе будет учитываться влияние уменьшения обучающей выборки.

Ожидание: после исправления протокола итоговая accuracy может несколько снизиться. Предварительно ожидается разница порядка 0,2–1,0 процентного пункта.

5. Улучшение и расширение

В качестве дополнительного эксперимента предлагается исследовать, сохраняется ли преимущество адаптированного stem при уменьшении размера модели.

Для этого будет использована уменьшенная конфигурация ConvNeXt-Tiny (около 3,3 млн параметров), которая сравнивается с основной конфигурацией Base (около 7,3 млн параметров).

Для каждой конфигурации будут рассмотрены варианты со стандартным и адаптированным stem.

Основная исследуемая величина:

$\Delta_{stem} = Accuracy_{adaptive} - Accuracy_{standart}$

Ожидание: уменьшенная модель продемонстрирует более низкую абсолютную точность, однако преимущество адаптированного stem сохранится. Величина прироста может зависеть от числа параметров и ёмкости модели.

Этот эксперимент является дополнительным и будет выполняться при наличии вычислительных ресурсов.

6. Бюджет и план

Время полного обучения будет оценено после профилирования первых 5–10 эпох.

Пусть T_B — время одной эпохи Base, T_T — время одной эпохи Tiny.

Тогда оценка времени полного запуска:

$$C_B \approx 300T_B, \qquad C_T \approx 300T_T.$$

Таблица 1: Предварительный план экспериментов

Эксперимент	Запуски	Эпохи
Оригинальный код	1	300
Base: полный и без модулей, 3 seed	6	300
Проверка корректной оценки	1–3	300
Tiny: scaling-эксперимент	2	300
GRN-only (опционально)	1	300

Оценка GPU-часов: по результатам проверочного запуска на NVIDIA Tesla T4 среднее время одной эпохи составило около 2.8 мин. Таким образом, полное обучение модели Base в течение 300 эпох потребует приблизительно 14 GPU-часов без учёта накладных расходов..

Проект выполняется индивидуально.

При недостатке ресурсов сначала исключается GRN-only, затем scaling-эксперимент. Приоритет сохраняется за воспроизведением основной модели и аудитом оценки.

7. Декларация использования LLM

При подготовке плана использовался ChatGPT (OpenAI) для разбора статьи, обсуждения экспериментального дизайна, анализа исходного кода и редактирования текста.

Проверка кода, запуск экспериментов, анализ полученных данных и формулирование итоговых выводов выполняются автором. Все численные результаты получены непосредственно в экспериментах.